DSProject5DS1 · branch orchestrator · commit 8feec36

# OpsAgents: how the agents work, and what is left

Five specialist agents, one orchestrator and one guardrail, sharing a single PostgreSQL database and a web dashboard. This report covers what works today, what does not, and how a trigger turns into an approved, audited action.

20 Oct 2026 | 149 tests pass | 18 skipped (drift) | 20/20 policy tests

01

## Status at a glance

Checked by running the app locally with the real models and clicking through every page of the dashboard.

| PART | STATE | DETAIL |
|---|---|---|
| Orchestrator | Working | Routing, triage, approvals, execution and summaries, all persisted in PostgreSQL. |
| Guardrail | Working | Risk scan, OPA policy and LLM judge. Self-approval and double votes are blocked. |
| Diagnostic / RCA | Working | DeepSeek-R1 found the `DATABASE_URL` cause on the sample log. |
| Code Reviewer | Working | Grok found 6 issues (4 high) on the leaky-PR demo and produced a lint patch. |
| Merge Assistant | Demo only | Works on a scratch repo it creates. Not connected to real GitHub branches. |
| Dependency Resolver | Degraded | OpenTofu is not installed, so it parses the `.tf` files itself. |

| PART | STATE | DETAIL |
|---|---|---|
| Drift Detection | Not running | Its `data` package was never committed to the repo. |
| GitHub watching | Untested | Built and unit-tested, never run against a real GitHub account. |
| Writing back to GitHub | Not built | Approved patches are dry runs. Nothing is pushed or commented. |
| User accounts | Not built | "Acting as" is a dropdown. No login. |
| Dashboard | Working | All 14 pages load with no console errors, in dark and light, down to phone width. |

02

## What is not working

Ordered by what would hurt a live demo first.

BLOCKS THE DEMO

**GitHub is not connected** blocker

The watcher, alerts and automatic investigations exist, but no account has been connected, so none of it has run against the real GitHub API. Real data can differ from what the tests assume, for example log download permissions on Actions runs.

**Fix:** create a GitHub OAuth App with Device Flow enabled and paste its Client ID in Settings, or paste a personal access token. Follow `hendch/DSProject5DS1`, push a commit that breaks a test, and check that the red alert and an RCA run appear within a minute.

**Drift Detection cannot start** blocker

Omar's repository ignored every folder called `data/`, which also hid the agent's own source package `agents/drift_detection/data`. Without it, the agent cannot load, and 18 of its tests are skipped.

**Fix:** Omar commits that package (and narrows the ignore rule), then runs `drift-agent init` once to train the first model.

### Dependency Resolver runs on its fallback degraded

It wants the OpenTofu CLI to build the real dependency graph. Without it, it reads the `.tf` files with a static parser. The Guardrail treats fallback output as less reliable and raises its risk one level. It also only runs when a trigger includes a folder of `.tf` files, which today means the demo scenario only.

**Fix:** install OpenTofu and put `tofu` on the PATH.

A BUYER WOULD NOTICE

### No real user accounts

Who you are is picked from the "Acting as" menu. The rules "you cannot approve what you triggered" and "two different people for high-risk production changes" work on names, not on verified identities. Anyone who can open the dashboard can approve as anyone. The API also accepts requests from any origin.

**Fix:** tie each person to their GitHub sign-in and restrict the API to the dashboard's origin.

### Nothing is written back to GitHub

An approved Code Reviewer patch is recorded as "Dry run: would push bot/minimal-fix and open a draft pull request". Setting `ORCHESTRATOR_GITHUB_PUBLISH=true` does not help yet: it reports failure because publishing was never implemented. Review findings are not posted as PR comments either.

**Fix:** push the patch branch and open the draft PR through the GitHub API with the signed-in token, and post findings as a PR review.

### Watching is by polling, and partial

GitHub cannot reach `localhost`, so the app asks GitHub every 60 seconds instead of receiving webhooks. It sees the 25 latest commits on the default branch, other branches only through their CI runs, and the 20 most recently updated PRs. A push with no CI and no PR is listed but not analysed.

### Merge and drift are not triggered by GitHub

Only failed CI and pull requests start runs automatically. The Merge Assistant runs on a demo repo unless the API is given a local repository path.

**Local only**

The app runs on one machine with an embedded PostgreSQL. It is not deployed. The free Hugging Face tier has rate limits; when a model fails, agents fall back to rules and the run is marked "degraded".

SMALL THINGS

**Agent registry shows the wrong model for Drift**

It shows the shared Llama profile. The drift agent actually reads `XAI_MODEL` and uses Grok.

**Settings and Agent registry load slowly**

They show loading placeholders for a few seconds on first open.

**Desktop notifications not tested**

The in-app preview browser blocks them. The in-page toasts work.

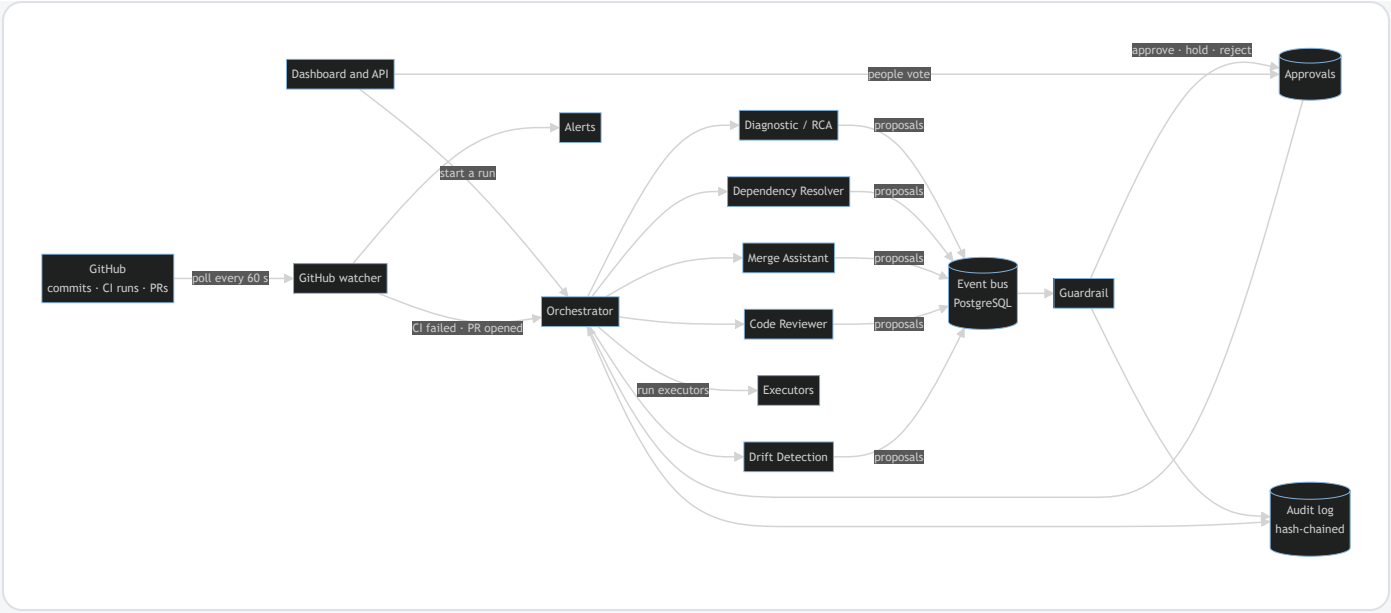
**Work is not pushed**

The `orchestrator` branch exists only on Rayen's machine. An older commit on `guardrail-agent` (2455c70) still carries an AI co-author line, which needs a force push to remove.

03

## How it fits together

Agents never call each other. The orchestrator calls each one through a small adapter, passes what earlier steps found to later steps, and publishes every proposal as an event. The Guardrail decides on every proposal before anything is executed, and every decision lands in a tamper-evident audit log.

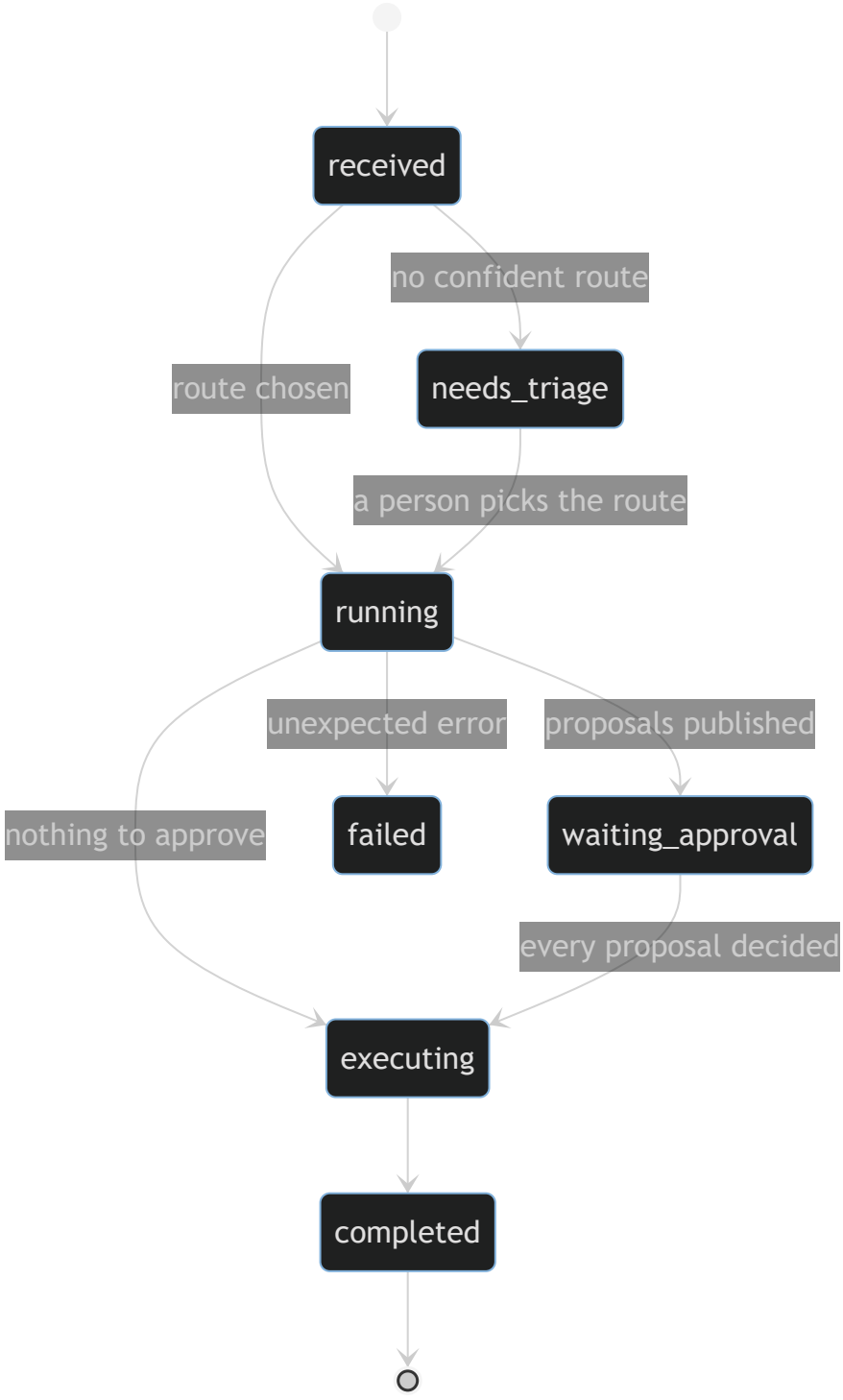



**Why this shape.** Each teammate's agent stays a separate module with its own fallback. The orchestrator is plain code, not an LLM, so a run always takes the same path for the same input and can be replayed from the database. LLMs are used where judgement helps: diagnosing, reviewing, explaining, and a second opinion in the Guardrail.

04

## Life of a run

A run is one trigger and everything that follows from it. Its state lives in `orchestrator.runs`, so a restart picks up where it stopped.




### Example: the failed IAM deploy

1. Sami starts the "Failed deploy: IAM cycle" scenario. The trigger is a deploy failure, so the rules pick the route `deploy_failed`: Dependency Resolver, then Diagnostic / RCA.
2. The Dependency Resolver finds a cycle between `aws_iam_role`, `aws_eks_cluster` and `aws_iam_role_policy` and proposes a fix. Like every agent, it labels its own risk "low".

3. The Guardrail does not trust that label. It sees `iam` in the payload, a permissions change, and raises the risk to high. The policy rule `high_always_human` holds it for one approval.
4. The RCA agent ranks the causes of the log. That is advice only, so the Guardrail approves it automatically.
5. Sami tries to approve the fix and gets refused: you cannot approve what you triggered. Ines approves it.
6. The executor records "skipped": the fix is an explanation that the owner applies by hand. The run completes, the orchestrator's model writes a short summary, and each step is in the audit log.

05

# Routing

Choosing which agents handle a trigger goes down a ladder, and stops at the first step that is sure enough.

| STEP | HOW IT DECIDES | WHEN IT IS USED |
|---|---|---|
| 1. Rules | The trigger type (failed CI, pull request, scenario), or keywords in the text such as "terraform", "merge", "drift". | Almost always. Every GitHub trigger is routed here. |
| 2. LLM triage | The orchestrator's model reads free text and returns a route and a confidence. | Free-text requests with no matching keyword. Accepted at confidence 0.7 or above. |
| 3. Human triage | The run waits on the Triage page until a person picks the route. | When the model is unsure or unavailable. |

### Routes

| ROUTE | STEPS, IN ORDER |
|---|---|
| `build_failed` | Diagnostic / RCA |
| `deploy_failed` | Dependency Resolver → Diagnostic / RCA |

| ROUTE | STEPS, IN ORDER |
|---|---|
| `merge_requested` | Merge Assistant |
| `pull_request` | Diagnostic / RCA *only if the PR's tests failed* → Code Reviewer |
| `drift_check` | Drift Detection → Diagnostic / RCA *only if drift looks like a pipeline bug* |

Conditional steps read the facts earlier steps left behind. For a failing PR, for example, the RCA diagnosis is passed to the Code Reviewer so its review can mention why the tests broke.

06

## The agents

Each agent can propose only the action types listed for it. The Guardrail rejects anything else.

### Diagnostic / RCA

zrharounesprit

Working

| | |
|---|---|
| Job | Explains why a build failed. |
| Input | A failed CI log, with secrets removed and cut to the end. |
| How | Strips colours and timestamps and keeps error lines with a little context (up to 3,000 characters). Searches the knowledge base once per distinct error line, keeping documents within cosine distance 0.3. Asks the model, with worked examples, for up to three causes, each with a confidence and the document it relies on. Citations the model invents are removed, causes below 0.2 confidence are dropped, and the request is retried if the model copies an example answer. |
| Proposes | `rank_causes`, advice only |
| Model | DeepSeek-R1-Distill-Qwen-7B, a reasoning model |
| Fallback | Ranks the error lines by severity and matches each to the closest document. |

## Dependency Resolver

Lyna

Degraded

| | |
|---|---|
| Job | Finds circular dependencies in OpenTofu / Terraform code and explains how to break them. |
| Input | A folder of `.tf` files. |
| How | Builds the resource graph with `tofu graph`, finds the cycles, classifies each one and has the model explain the fix. |
| Proposes | `propose_fix` |
| Model | Llama 3.1 8B Instruct |
| Fallback | A static parser instead of OpenTofu and a keyword classifier instead of the model. The run is marked degraded. |

## Merge Assistant

Muaad

Demo only

| | |
|---|---|
| Job | Checks whether two branches merge cleanly and proposes how to resolve conflicts. |
| How | Tries the merge in a scratch copy, never in the real repo, then describes each conflict and a resolution. It never merges. |
| Proposes | `flag_conflict` |
| Model | Llama 3.1 8B Instruct |
| Fallback | A rule-based description of the conflicting lines. |

## Code Reviewer

hendch — Working

| | |
|---|---|
| Job | Reviews pull requests for ML and MLOps mistakes such as data leakage, and proposes safe lint fixes. |
| How | Clones the PR read-only, using the token for that one command only. Runs `ruff`, then runs the reviewer in its own process. The model uses tools to read the code, and returns findings by severity plus a minimal patch. |
| Proposes | `review_code` (advice only) and `propose_patch` (a draft PR, currently a dry run) |
| Model | Grok 4.20, non-reasoning, through xAI |
| Fallback | A scripted review based on the lint results. |

## Drift Detection

omar — Not running

| | |
|---|---|
| Job | Watches a deployed model for data and concept drift. |
| How | Scores each new batch of production data and sets a severity. When drift is found, the model investigates and recommends retraining, promoting or rolling back a model version. Its data lives in the `drift` schema of the shared database. |
| Proposes | `drift_report` (advice only), and `promote_model` and `rollback_model` (production, high risk, so two approvals) |
| Model | Grok 4.20, reasoning, through xAI |
| Blocked by | The missing `data` package (see section 02). |

## Orchestrator

shared — Working

| | |
|---|---|
| Job | Turns triggers into runs, calls agents in order, collects decisions, runs executors and writes the summary. |
| How | A state machine stored in PostgreSQL. A background sweep advances runs that are waiting, and on startup any interrupted run is recovered. |
| Model | Llama 3.1 8B Instruct, used only for triage of free text and the run summary. |
| Fallback | Human triage, and a plain template summary. |

07

# The Guardrail

Every proposal is checked in three layers. The rule that holds them together: each layer can make a decision stricter, and none can make it looser.

### Layer 1: risk from evidence

The proposal starts at the risk the agent declared. The Guardrail then scans what the action actually touches and raises the level when it finds:

| SIGNAL IN THE PAYLOAD | RAISES TO |
|---|---|
| `prod`, `production` | high |
| Destructive words: destroy, delete, drop table, truncate, `rm -rf`, terminate | high |
| Permissions and security: IAM, KMS, secrets, security groups, firewalls, roles | high |
| Billable infrastructure: GPUs, instances, RDS, clusters, NAT gateways, autoscaling | medium |
| Infrastructure state: backend, tfstate, state lock | medium |
| The agent fell back to rules or static analysis | one level up |

Advice-only actions skip this scan, because approving them changes nothing.

### Layer 2: the policy

Written in Rego and evaluated by Open Policy Agent. Rules are tried from top to bottom and the first match wins.

| RULE | WHEN | DECISION |
|---|---|---|
| `agent_not_allowed_action` | The agent proposed an action it is not allowed to | reject |
| `prod_high_two_person` | High risk in production | hold · 2 people |
| `high_always_human` | High risk anywhere else | hold · 1 person |

| RULE | WHEN | DECISION |
|---|---|---|
| `advisory_only` | Advice: rank_causes, review_code, drift_report | auto-approve |
| `prod_requires_human` | Any other change in production | hold · 1 person |
| `staging_medium_human` | Medium risk in staging | hold · 1 person |
| `low_risk_auto` | Low risk in dev or staging | auto-approve |
| `dev_medium_auto` | Medium risk in dev | auto-approve |
| `default_hold` | Nothing matched | hold · 1 person |

### Layer 3: the LLM judge

For changes the policy did not reject, a separate model reads the proposal and the evidence and says whether it looks risky. It can turn an automatic approval into a hold (rule `llm_escalation`), never the reverse. If the model is unreachable, the policy decision stands. The judge is Qwen 2.5 7B, a different model family from the agents it checks.

### Approvals

- The person who triggered a run cannot approve its proposals (the API answers 403).
- A person cannot vote twice on the same proposal (409).
- Pending approvals expire after 4 hours and count as rejected.
- Every decision is written to `reasoning_logs`. Each row includes the hash of the row before it, so changing or deleting a row breaks the chain. The Audit page checks the chain and shows whether it is intact.

08

## After approval

Each decided proposal goes to an executor once. The result is stored, so running the executors again never repeats an action.

| ACTION | WHAT HAPPENS WHEN APPROVED |
| --- | --- |
| `rank_causes, review_code, drift_report` | not needed Advice only. |
| `propose_fix` | skipped The fix is an explanation. The owner applies it by hand. |
| `flag_conflict` | skipped A person merges. The assistant never does. |
| `propose_patch` | dry run Records the branch and draft PR it would open. Real publishing is not built. |
| `promote_model, rollback_model` | done The drift agent applies it (once the agent can run). |
| Anything rejected or expired | skipped Nothing is executed. The drift agent is told about rejections. |

09

# GitHub

## Connecting

In Settings, "Sign in with GitHub" uses GitHub's device flow: the app shows a short code, you approve it on github.com, and the app receives a token. This needs a GitHub OAuth App, created once, with Device Flow enabled. The other way is to paste a personal access token. Then you choose which repositories to follow on the Repositories page.

## What it reacts to

| ON GITHUB | IN OPSAGENTS |
| --- | --- |
| A commit's CI passes | Green status on the commit in Activity |
| A commit's CI fails | Critical alert. The failing job's log goes to the RCA agent automatically. |
| CI passes again after a failure | "CI is green again" alert |

| ON GITHUB | IN OPSAGENTS |
|---|---|
| A PR is opened or gets new commits | Info alert. The Code Reviewer reviews it automatically, with RCA first if its tests failed. |
| Token rejected, rate limit hit, repo unreadable | Alert that points to Settings or Repositories |

The first sync of a repository only records its history, so old failures do not raise alerts. Alerts are de-duplicated, so a failure is reported once. Automatic investigation can be turned off in Settings. Logs have secrets removed before they are stored or sent to a model.

10

## Models

Every agent has its own model profile in `.env` (`<AGENT>_LLM_MODEL` and related settings) and falls back to the shared default. All models are called through one OpenAI-compatible client.

| USED BY | MODEL | PROVIDER | WHY THIS ONE |
|---|---|---|---|
| Orchestrator, Dependency Resolver, Merge Assistant | `Llama-3.1-8B-Instruct` | Hugging Face | Fast and cheap. Their tasks are short: classify, explain, summarise. |
| Diagnostic / RCA | `DeepSeek-R1-Distill-Qwen-7B` | Hugging Face | A reasoning model. Finding a root cause means weighing log lines against documents step by step. |
| Guardrail judge | `Qwen2.5-7B-Instruct` | Hugging Face (Featherless) | A different family from the agents it checks, so it is less likely to share their blind spots. Reliable at structured JSON. |
| Code Reviewer | `grok-4.20-0309-non-reasoning` | xAI | The reviewer needs tool calls. The Hugging Face coder models tried did not support them. |

| USED BY | MODEL | PROVIDER | WHY THIS ONE |
|---|---|---|---|
| Drift Detection | `grok-4.20-0309-reasoning` | xAI | Omar's original setup. |

### When a model fails

- Each call is retried twice. After a failure, that model is skipped for 30 seconds so a dead provider does not slow every run.
- When a model is unavailable, the agent switches to its rule-based fallback, and the step is marked "degraded" on the run page.
- Reasoning output ( `<think>` blocks) is stripped, and JSON is extracted even when a model wraps it in prose. RCA turns JSON mode off, because DeepSeek-R1 answered with nothing in JSON mode.

11

## Data and plumbing

Everything lives in one PostgreSQL database, set up by numbered migrations `001` to `006` ( `python -m shared.migrate` ).

| WHERE | WHAT |
|---|---|
| `events`, `event_receipts` | The event bus. Publishing and logging happen in one transaction. Consumers are woken by LISTEN/NOTIFY and keep one receipt per event. A failed event is retried 30 seconds later, with growing delays, and marked dead after 5 attempts. |
| `approvals` | Guardrail decisions and votes |
| `reasoning_logs` | The hash-chained audit log, append-only |
| `orchestrator.*` | Runs, their steps and their actions |
| `knowledge.documents` | The RCA knowledge base, as pgvector embeddings (BAAI bge-small-en-v1.5, 384 dimensions) |
| `github.*` | Followed repos' commits and PRs, plus what has already been alerted on |

| WHERE | WHAT |
| --- | --- |
| `alerts`, `app_settings` | Notifications and settings such as the OAuth Client ID |
| `drift` | The drift agent's batches, runs and model versions |

For development and tests, `pgserver` starts an embedded PostgreSQL 16 with pgvector, so no installation is needed. The policy runs on a local OPA binary in `.tools/`.

12

## Running it

Once, in the project folder:

```
pip install -e ".[dev]"
```

Start everything (database, policy engine, agents, GitHub watcher, dashboard) and open localhost:8080:

```
python -m app
```

Tests:

```
pytest -q
```

Add `?mock=1` to the dashboard URL to try the interface with sample data and no backend.

---

Written from the code on branch `orchestrator` (commit 8feec36) and a local run on 20 October 2026.